

Amazon Cloud Storage Services —

Overview

This tutorial introduces **AWS storage services** and how they integrate with **Amazon EC2**, covering:

- **Block storage** (EBS)
- **Network file systems** (EFS)
- **Object storage** (S3)
- **Large-scale data transfer services**
- **Secure access to S3 from EC2 using IAM roles**

Storage Types in AWS

Block Storage

- Data is stored in fixed-size blocks.
- Only modified blocks are updated on write operations.
- Typically formatted by the OS into a filesystem.
- Example use case: operating system disks, databases.

Filesystem Storage

- Built on block storage.
- Organizes data in directories and files.
- Can be local or shared over a network.

Object Storage

- Stores data as objects (data + metadata).
- Entire object is replaced on modification.
- Highly scalable and accessed via HTTP APIs.



Distributed Storage Interface Types

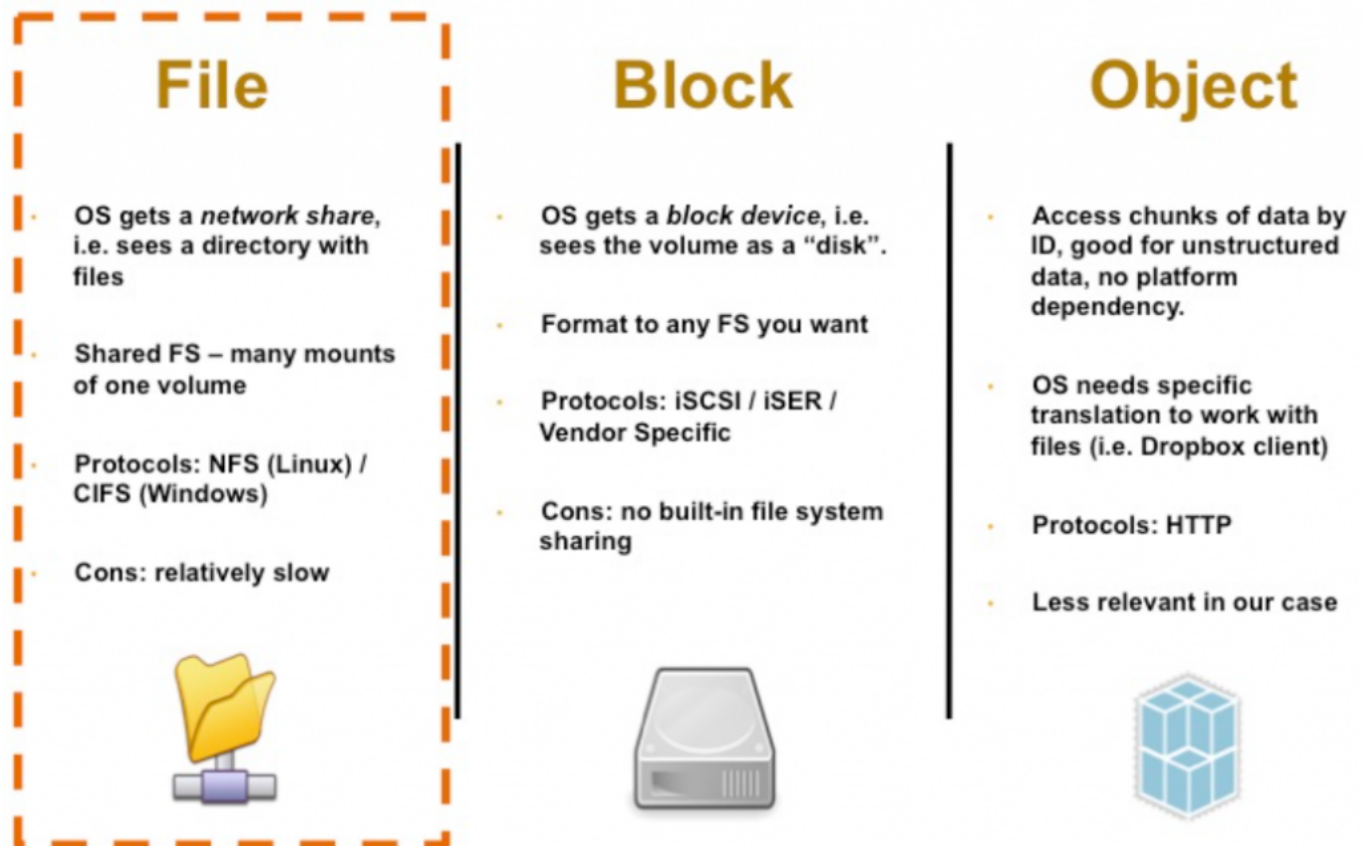


Figure 2.1: Relationship between block storage, filesystems, and object storage.

Amazon Elastic Block Store (EBS)

Key Characteristics

- Block-level storage for EC2 instances.
- Volumes are created independently and attached to EC2.
- Supports dynamic resizing.
- Commonly used as primary storage for EC2.

Workflow: Creating and Using an EBS Volume

1. Check existing disks on EC2

- Use Linux commands:
 - `sudo su`
 - `fdisk -l`

```
# fdisk -l /dev/sda

Disk /dev/sda: 68.7 GB, 68719476736 bytes
255 heads, 63 sectors/track, 8354 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk identifier: 0x00056b1c
```

Device	Boot	Start	End	Blocks
/dev/sda1	*	1	8167	65601396
/dev/sda2		8168	8354	1502077+
/dev/sda5		8168	8354	1502046

```
Command (m for help): n
All primary partitions are in use.
Adding logical partition 6
First sector (1245165-10288994, default 1245165): 1235164
Value out of range.
First sector (1245165-10288994, default 1245165): 1245165
Last sector, +/-sectors or +/-size{K,M,G,T,P} (1245165-10288994, default 10288994): +500M

Created a new partition 6 of type 'Linux' and of size 512 MiB.

Command (m for help): p
Disk /dev/sda: 931.53 GiB, 1000204886016 bytes, 1953525168 sectors
Disk model: Portable SSD T5
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 33553920 bytes
Disklabel type: dos
Disk identifier: 0xd13fa933
```

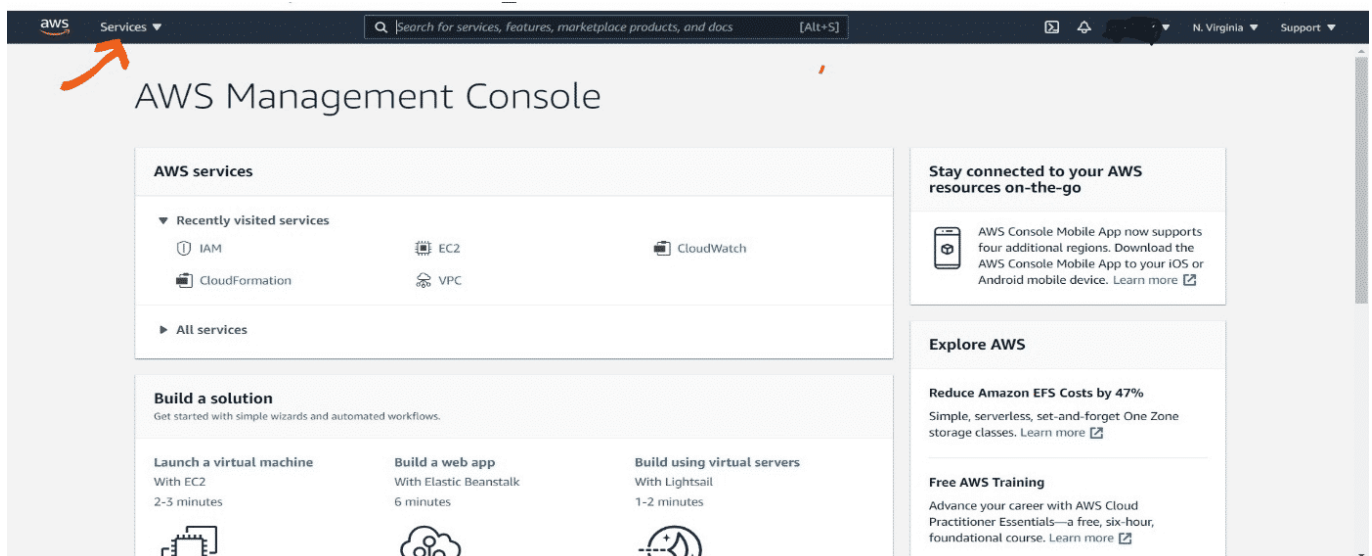
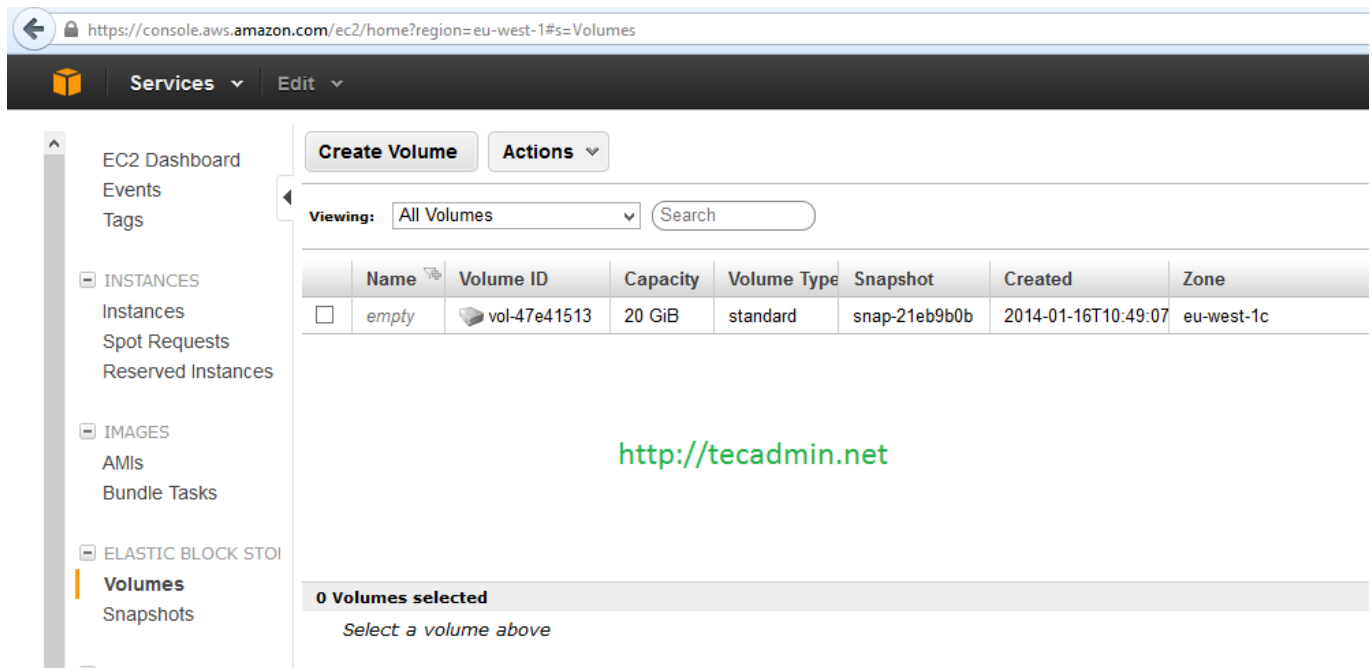
Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	*	1952501760	1953523711	1021952	499M	ef	EFI (FAT-12/16/32)
/dev/sda2		1950453760	1952501759	2048000	1000M	83	Linux
/dev/sda3		1704693760	1950453759	245760000	117.2G	83	Linux
/dev/sda4		65536	10288994	10223459	4.9G	5	Extended
/dev/sda5		131071	1179629	1048559	512M	83	Linux
/dev/sda6		1245165	2293724	1048560	512M	83	Linux

Partition table entries are not in disk order.

Figure 2.2: Existing root disk visible on the Linux instance.

2. Create an EBS volume

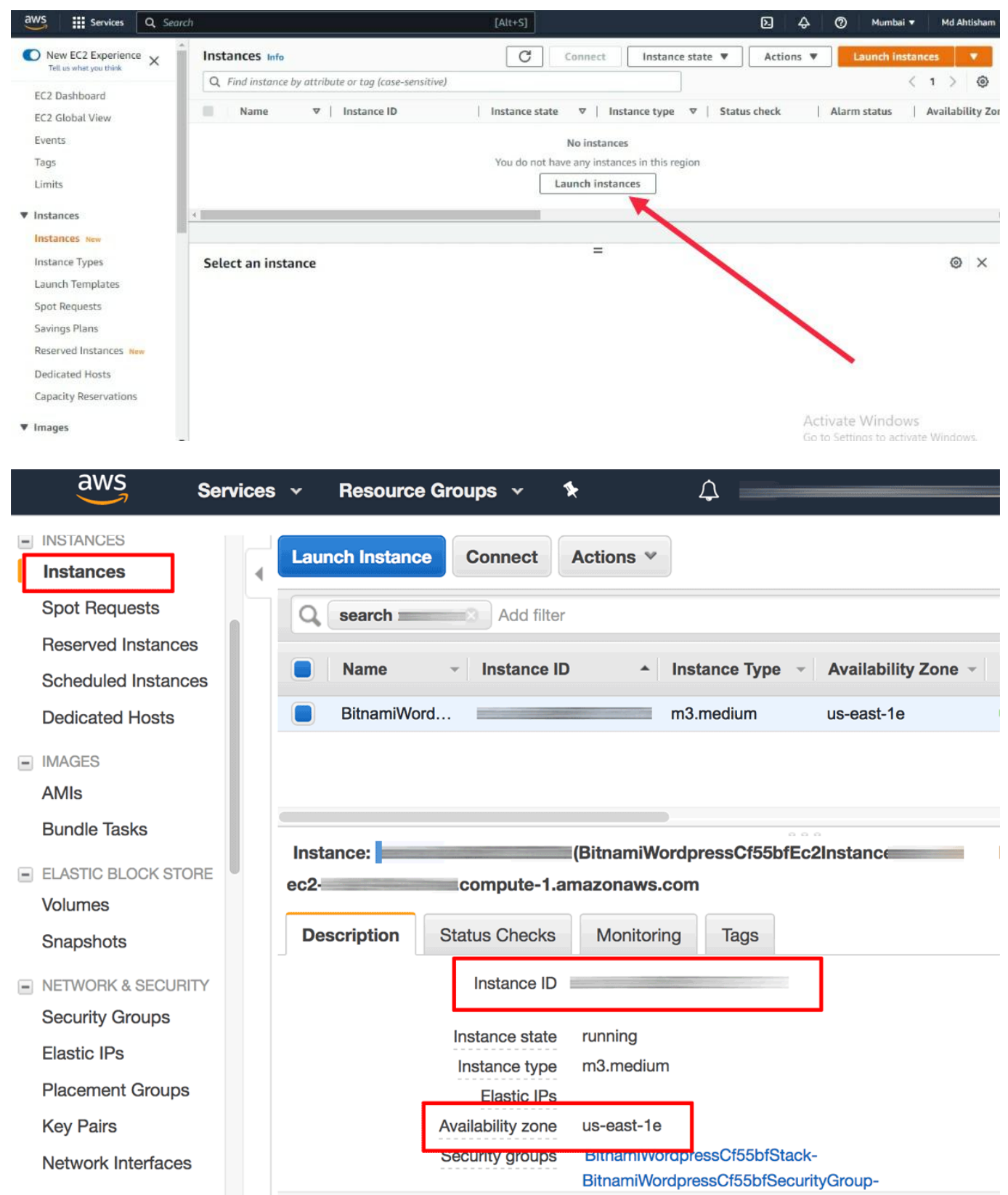
- AWS Console → EC2 → Elastic Block Store → Volumes → Create Volume
- Choose:
 - Volume size
 - Volume type (SSD, HDD)
 - Availability Zone



Figures 2.3–2.4: Viewing existing volumes and creating a new EBS volume.

3. Attach the volume to EC2

- Actions → Attach Volume
- Select target EC2 instance



Figures 2.5–2.6: Attaching the new EBS volume to the instance.

4. Format and mount the disk

- Detect disk using `fdisk -l`
- Create filesystem: `mkfs`
- Mount to a directory using `mount`


```
[root@Tecmint installer]# fdisk -l
```

Disk /dev/xvda: 107.4 GB, 107374182400 bytes, 209715200 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x000969af

Device	Boot	Start	End	Blocks	Id	System
/dev/xvda1	*	2048	1026047	512000	83	Linux
/dev/xvda2		1026048	209715199	104344576	8e	Linux LVM

Disk /dev/mapper/rhel-root: 104.7 GB, 104681439232 bytes, 204455936 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/mapper/rhel-swap: 2164 MB, 2164260864 bytes, 4227072 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/xvdc: 21.5 GB, 21474836480 bytes, 41943040 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

```
Disk /dev/sda: 100 GiB, 107374182400 bytes, 209715200 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x16a5d2c8
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	*	2048	976895	974848	476M	83	Linux
/dev/sda2		978942	209713151	208734210	99.5G	5	Extended
/dev/sda5		978944	8976383	7997440	3.8G	82	Linux swap / Solaris
/dev/sda6		8978432	28977151	19998720	9.5G	83	Linux
/dev/sda7		28979200	209713151	180733952	86.2G	83	Linux

```
Disk /dev/sdb: 3.7 GiB, 4004511744 bytes, 7821312 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x0015a216
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sdb1	*	2048	7821311	7819264	3.7G	c	W95 FAT32 (LBA)

```
ubuntu@ubuntu1804:~$
```

Figure 2.7: New EBS volume formatted and mounted.

5. Resize EBS volume

- Increase volume size in AWS Console.
- Extend filesystem in the guest OS.

Amazon Elastic File System (EFS)

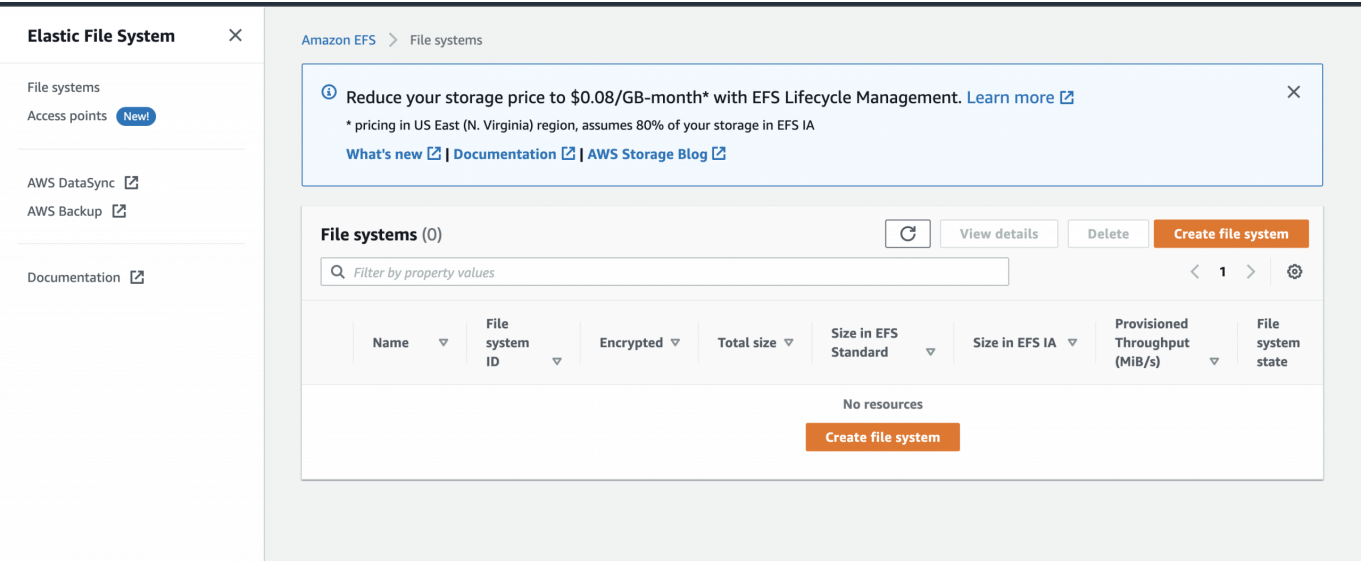
Key Characteristics

- Fully managed, elastic, shared filesystem.
- Accessible by multiple EC2 instances simultaneously.
- Uses NFS protocol.
- No need to predefine storage size.

Workflow: Creating and Using EFS

1. Create an EFS filesystem

- AWS Console → EFS → Create file system
- Choose:
 - One Zone (single AZ)
 - Standard (multi-AZ, higher durability)



Elastic File System

File systems

Access points

AWS DataSync

AWS Backup

Documentation

Amazon Elastic File System

Scalable, elastic, cloud-native NFS file system

Amazon Elastic File System (Amazon EFS) provides a simple, scalable, elastic file system for general purpose workloads for use with AWS Cloud services and on-premises resources.

What is Amazon Elastic File System?

aws

Amazon Elastic File System - Scalable, Elastic, Clo...

Copy link



Amazon Elastic File System (Amazon EFS) provides a simple, scalable, fully managed elastic NFS file system for use with AWS Cloud services and on-premises resources. It is built to scale on demand to petabytes without disrupting applications, growing and shrinking automatically as you add and remove files, eliminating the need to provision and manage capacity to accommodate growth. Amazon EFS is designed to provide massively parallel shared access to thousands of Amazon EC2 instances, enabling your applications to achieve high levels of aggregate throughput and IOPS with consistent low latencies.

Benefits and features

Dynamic elasticity

Fully managed

Create file system

Create an EFS file system with service recommended settings.

Create file system

Pricing (US)

Standard storage	\$0.30 per GB
Infrequent Access storage	\$0.025 per GB
Infrequent Access requests	\$0.010 per GB transferred
Provisioned Throughput	\$6.00 per MB/s

[AWS Pricing Calculator](#)

Get started

[What is Amazon Elastic File System?](#)

[Get started](#)

[How it works](#)

More resources

[Documentation](#)

[FAQ](#)

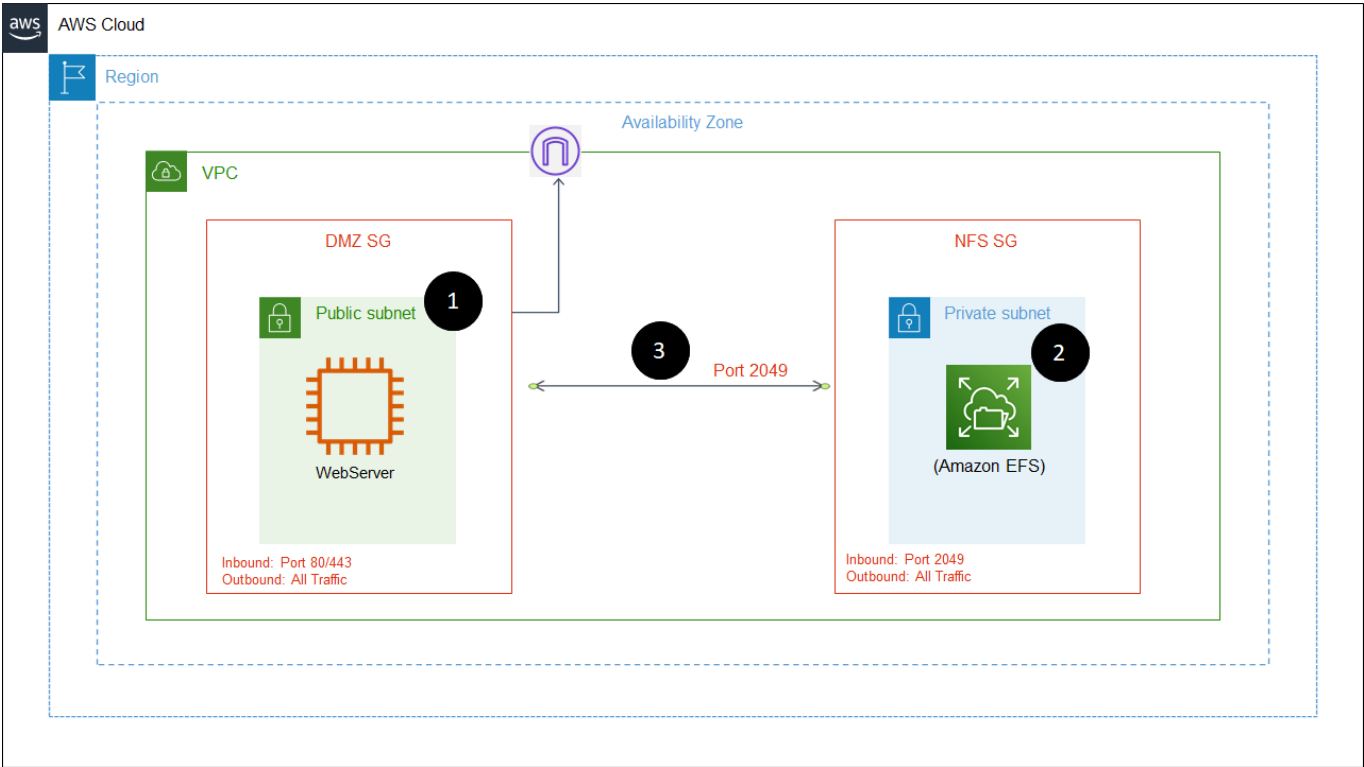
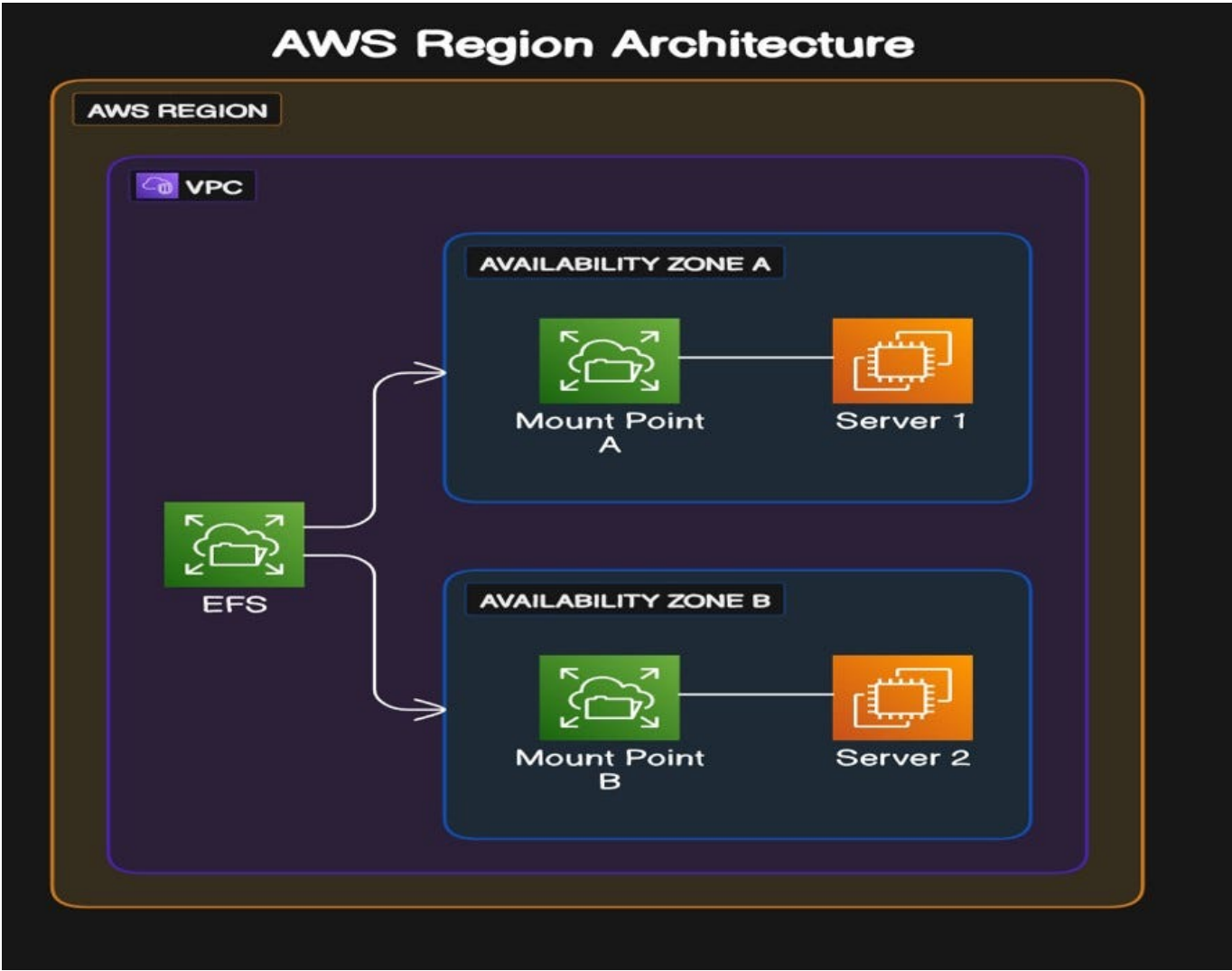
[EFS forum](#)

[Contact us](#)

Figures 2.8–2.9: Creating an EFS filesystem.

2. Mount EFS on a Linux EC2 instance

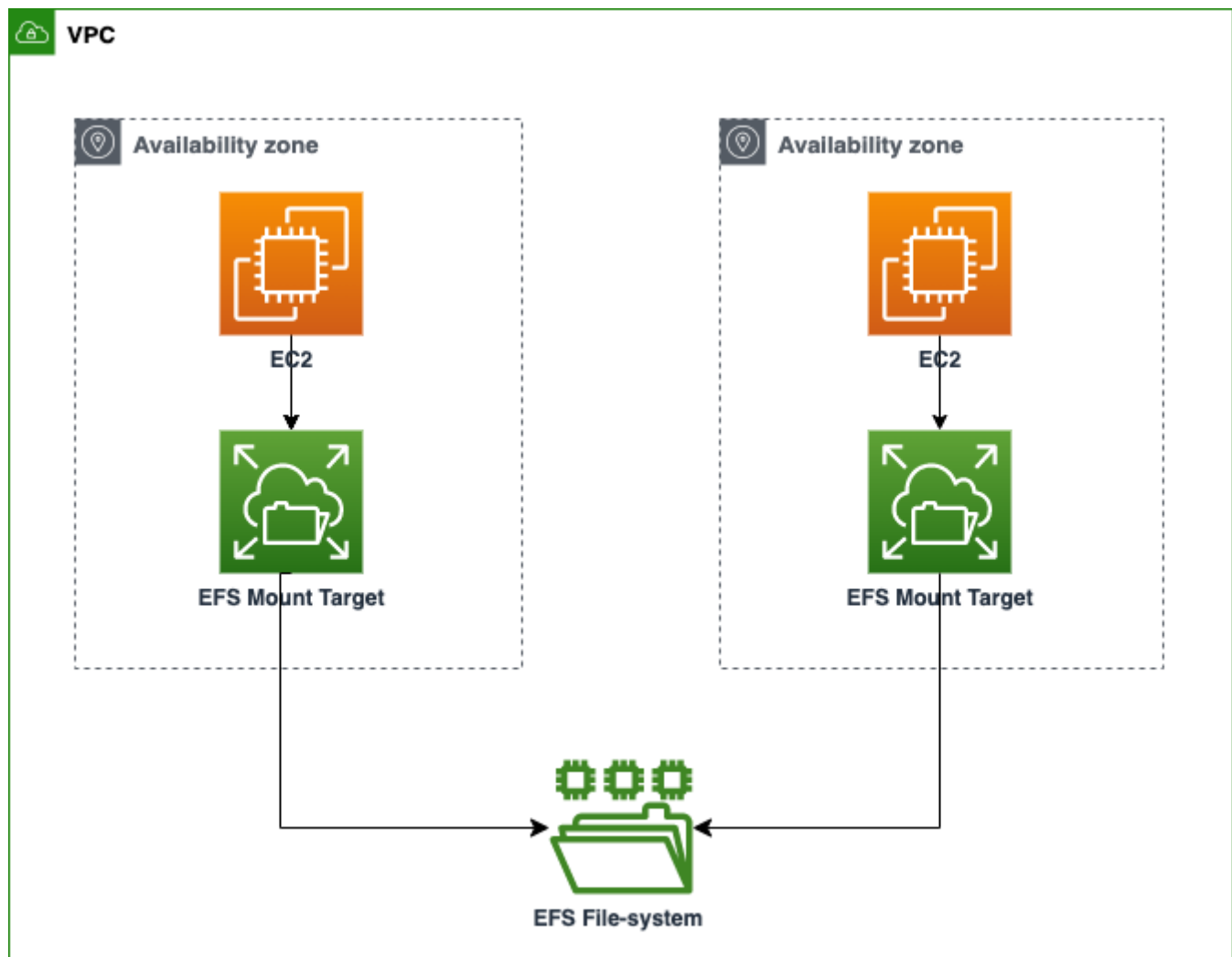
- Use NFS mount command provided by AWS.
- Mount to a local directory.



Figures 2.10–2.12a: Attaching EFS to a Linux instance.

3. Verify shared access

- Mount the same EFS on another EC2 instance.
- Files are readable and writable across instances.



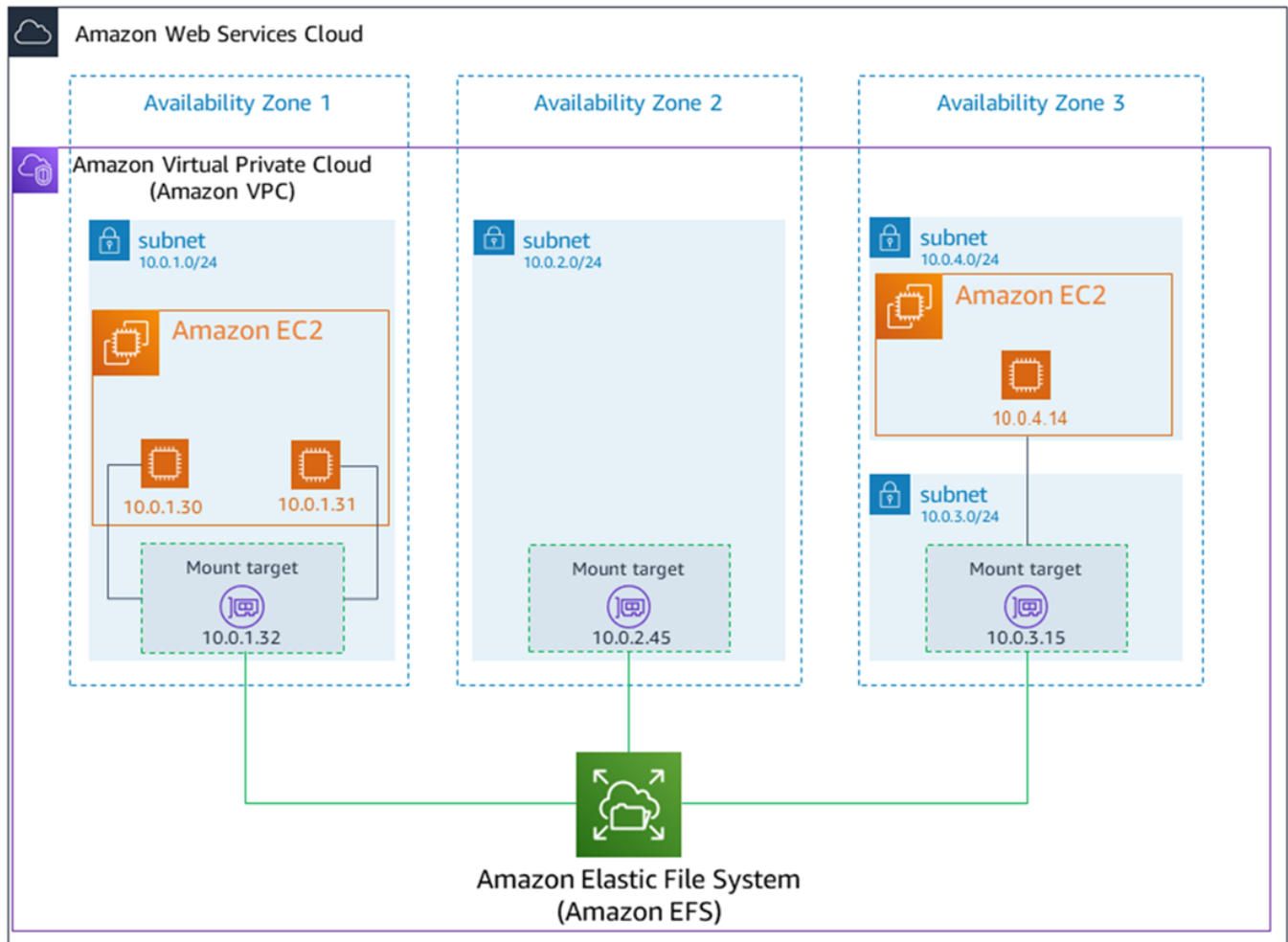


Figure 2.12: Shared EFS across multiple EC2 instances.

Amazon Simple Storage Service (S3)

Core Concepts

- Object-based storage.
- Globally accessible via public endpoints.
- Data stored in **buckets**.
- Bucket names must be globally unique.
- Object size: **0 bytes to 5 TB**.

S3 Storage Classes

- **S3 Standard** – high availability and durability.
- **S3 Standard-IA** – lower cost for infrequent access.
- **Reduced Redundancy Storage** – lower durability.
- **S3 Intelligent-Tiering** – automatic tiering.
- **Glacier** – archival storage with very high durability.

Additional Features

- **Versioning:** Maintains multiple versions of objects.

- **Lifecycle management:**

- Automatically transitions objects between storage classes.
- Supports expiration and deletion rules.

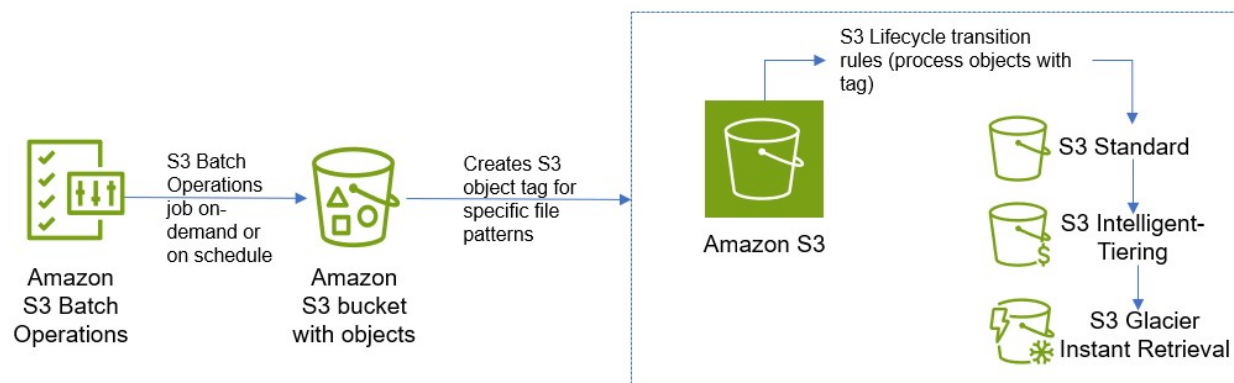
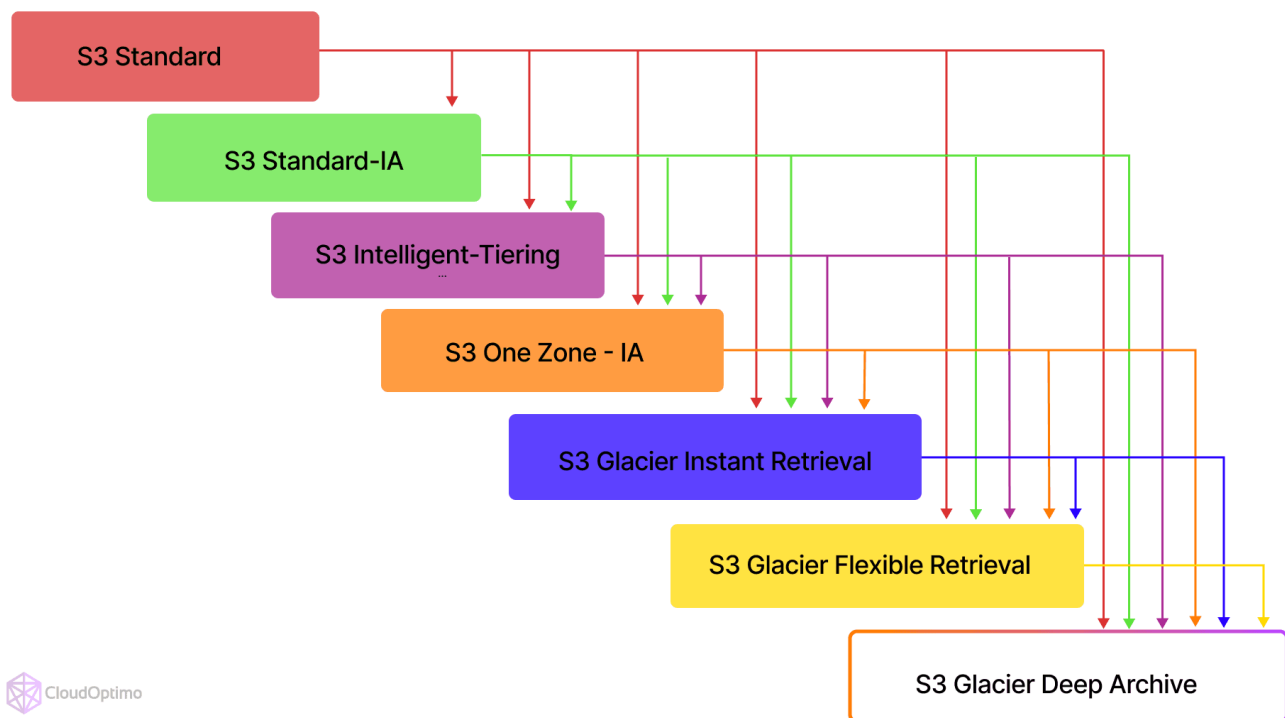


Figure 2.13: Example of S3 lifecycle transition policy.

Large-Scale Data Transfer Services

AWS Snowcone

- Small, rugged edge device.
- Supports offline and online data transfer.
- Suitable for edge computing.

AWS Snowball

- Appliance-based transfer service.

- Up to **80 TB** usable storage.
- Data encrypted using AWS KMS.

AWS Snowmobile

- Truck-based solution.
 - Up to **100 PB** per transfer.
 - Designed for massive enterprise migrations.
-

Accessing S3 from EC2 Instances

Problem

- EC2 instances cannot access S3 by default, even with a public IP.

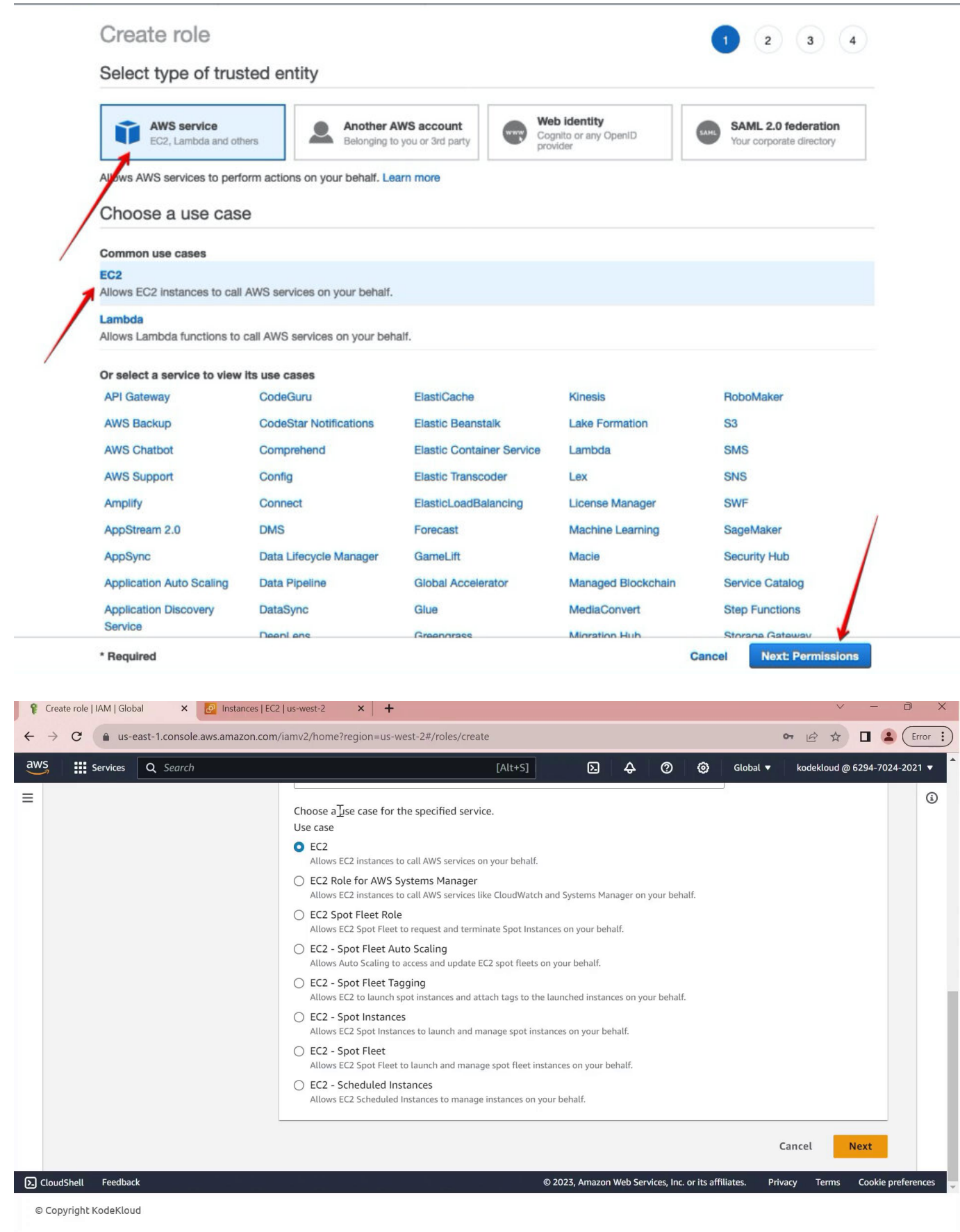
Solution: IAM Roles for EC2

- Assign an IAM role with S3 permissions to the EC2 instance.
- Applications and CLI on EC2 inherit role permissions.

Workflow: Granting S3 Access

1. Create IAM role

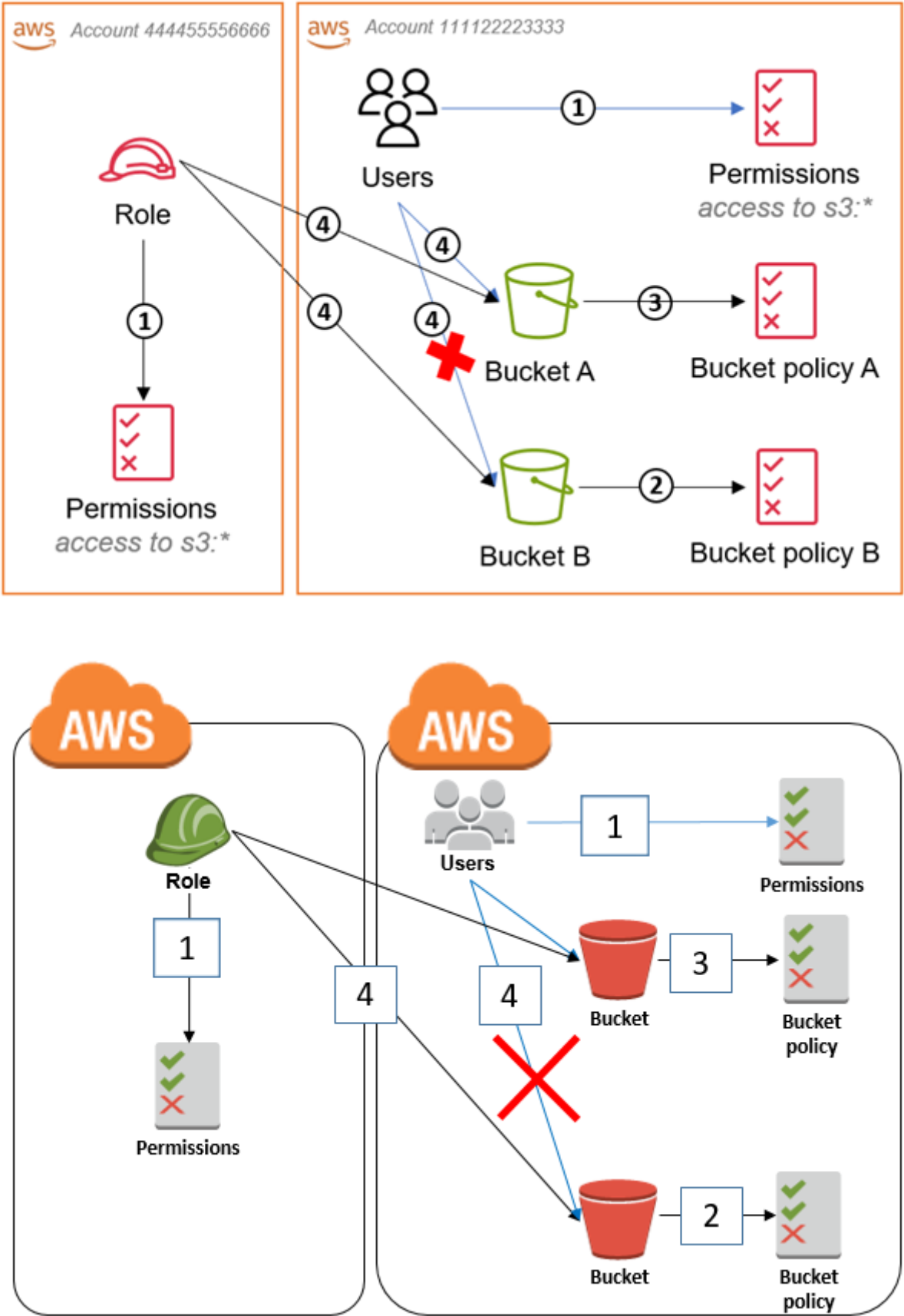
- IAM → Create role
- Trusted entity: **AWS service**
- Use case: **EC2**



Figures 2.15–2.16: Creating an EC2 IAM role.

2. Attach S3 permissions

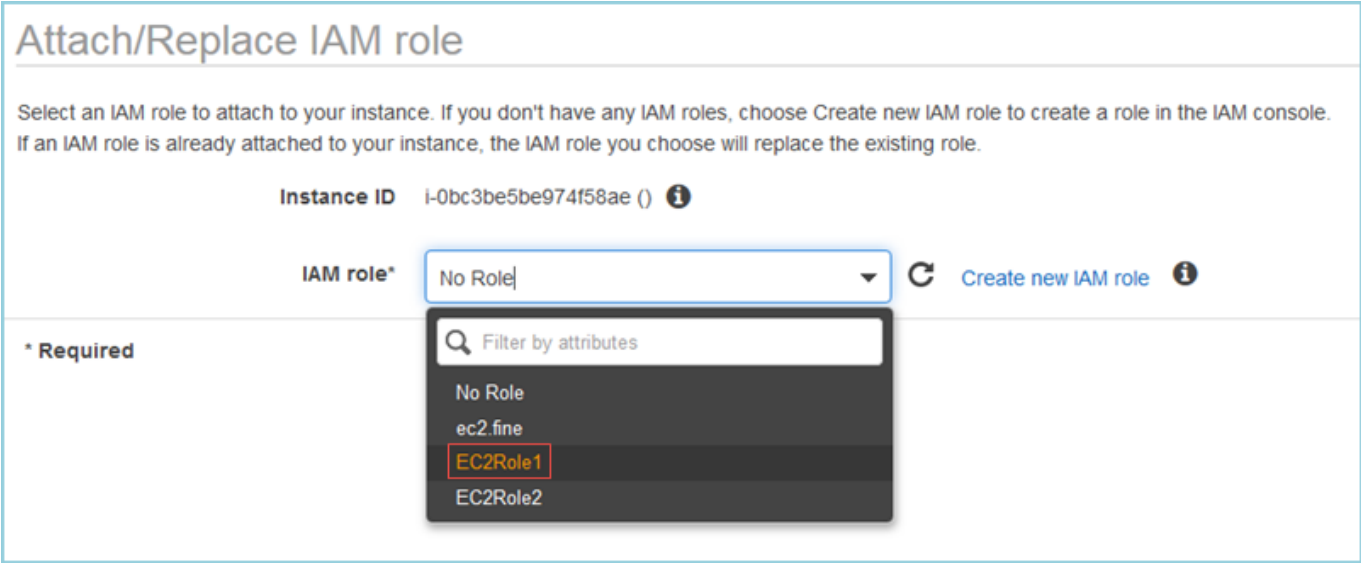
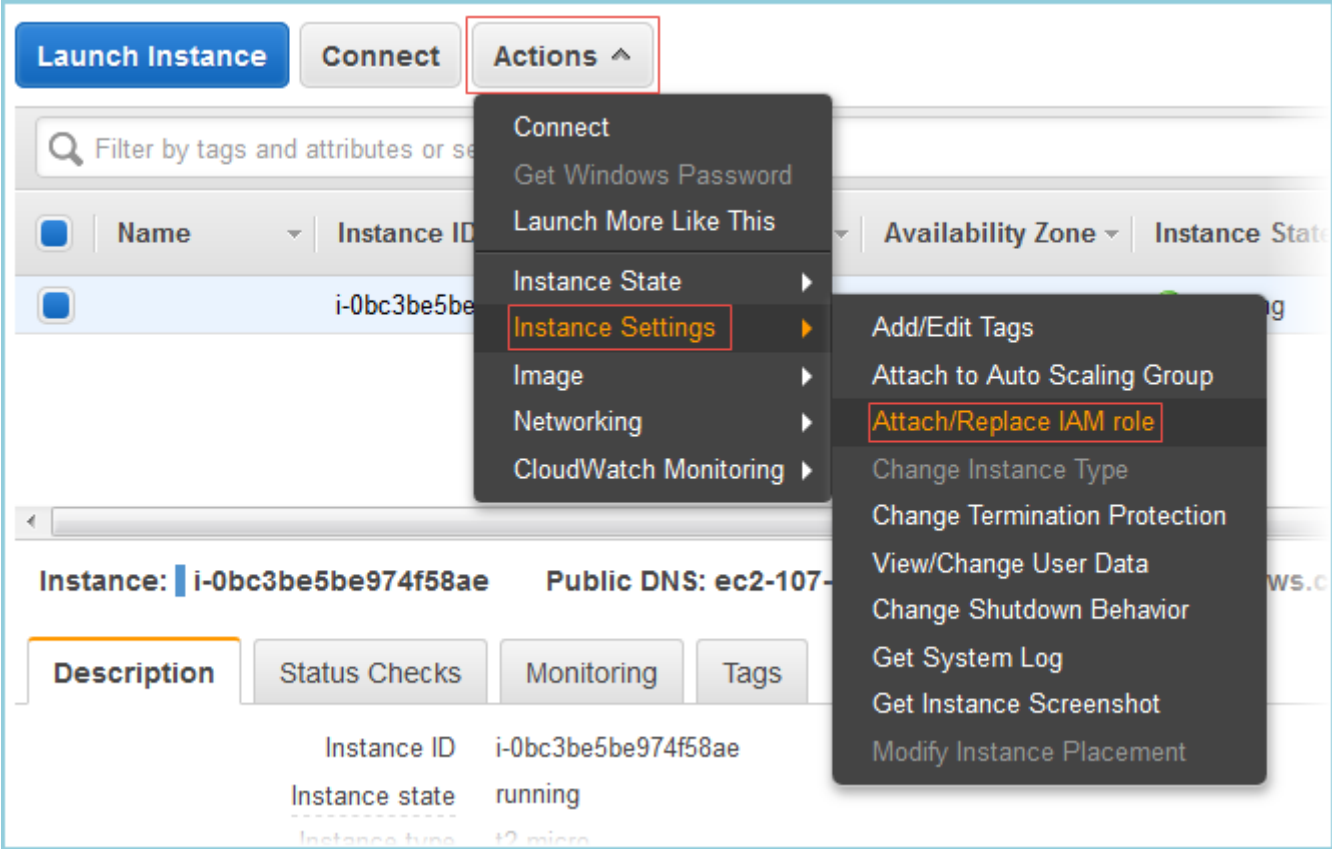
- Add policy: `AmazonS3FullAccess`



Figures 2.17–2.18: Assigning permissions and creating the role.

3. Attach role to EC2 instance

- EC2 → Instance → Actions → Security → Modify IAM role

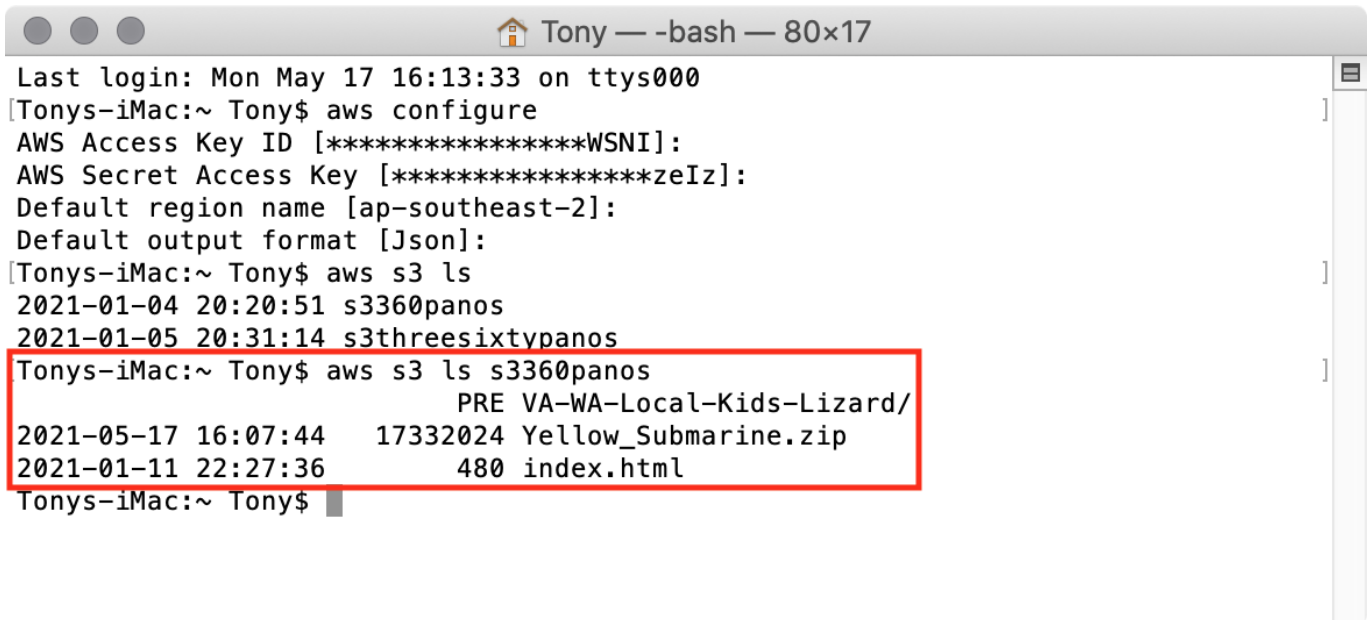


Figures 2.19–2.20: Attaching IAM role to EC2.

4. Verify access

- Run:

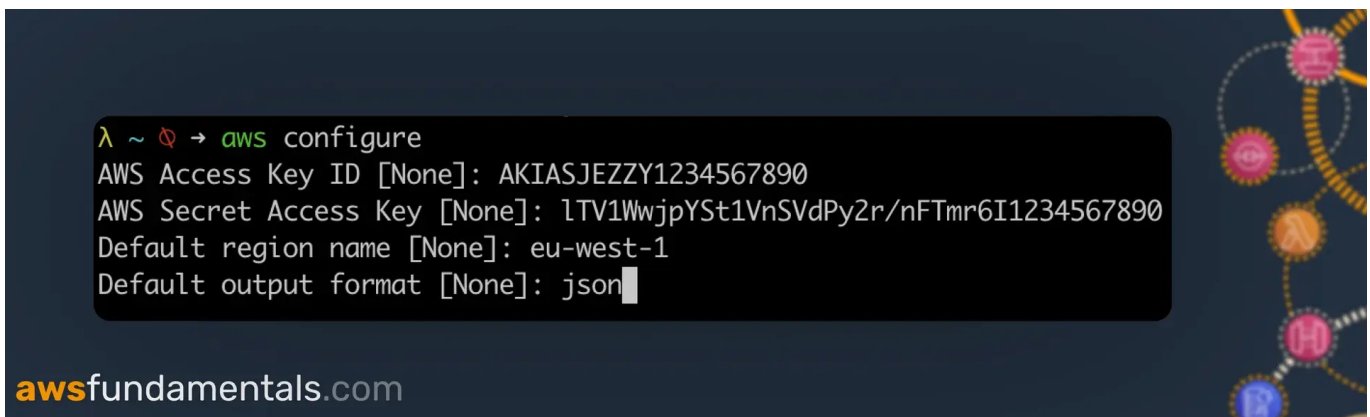
```
aws s3 ls
```



```

Tony — -bash — 80x17
Last login: Mon May 17 16:13:33 on ttys000
[Tonys-iMac:~ Tony$ aws configure
AWS Access Key ID [*****WSNI]:
AWS Secret Access Key [*****zeIz]:
Default region name [ap-southeast-2]:
Default output format [Json]:
[Tonys-iMac:~ Tony$ aws s3 ls
2021-01-04 20:20:51 s3360panos
2021-01-05 20:31:14 s3threesixtypanos
Tonys-iMac:~ Tony$ aws s3 ls s3360panos
                PRE VA-WA-Local-Kids-Lizard/
2021-05-17 16:07:44    17332024 Yellow_Submarine.zip
2021-01-11 22:27:36      480 index.html
Tonys-iMac:~ Tony$

```



```

λ ~ → aws configure
AWS Access Key ID [None]: AKIASJEZZY1234567890
AWS Secret Access Key [None]: lTV1WwjpYSt1VnSVdPy2r/nFTmr6I1234567890
Default region name [None]: eu-west-1
Default output format [None]: json

```

awsfundamentals.com

Figure 2.21: Successful listing of S3 buckets from EC2.

Chapter Summary

- **EBS:** Block storage for EC2, ideal for low-latency and persistent workloads.
- **EFS:** Shared filesystem for multiple EC2 instances.
- **S3:** Highly scalable, durable object storage.
- **Snowcone/Snowball/Snowmobile:** Physical data migration services.
- **IAM roles:** Secure, credential-free access to S3 from EC2.

These services collectively address diverse storage and data transfer requirements in AWS.

Reference Links

- <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html>
- <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/modify-ebs-volume-on-instance.html>
- <https://www.bu.edu/comtech/students/technical-guides/hardware/how-to-format-hard-drives/>